

ME315 - Lab03

Matheus Rodrigues (259256)

Dados relacionais

```
library(readr)
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

`filter`, `lag`

The following objects are masked from 'package:base':

`intersect`, `setdiff`, `setequal`, `union`

1. Importe, utilizando o pacote readr, cada um dos três arquivos disponíveis. Os objetos resultantes devem ser chamados flights, airlines e airports.
 - Para o arquivo de aeroportos, importe apenas as colunas IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE;
 - Para o arquivo de vôos:
 - a. Importe apenas as colunas DESTINATION_AIRPORT e ARRIVAL_DELAY;
 - b. Leia apenas 1 milhão de linhas por vez;
 - c. Remova vôos em que o aeroporto de destino comece com a letra 1;
 - d. Remova registros em que existam pelo menos uma coluna faltante;

- e. Determine, para cada parte do arquivo, as estatísticas suficientes para a determinação do atraso médio por aeroporto de destino;
- f. Ao finalizar a leitura do arquivo, combine as estatísticas suficientes de modo a produzir a média de atraso por aeroporto.

```
airlines <- read_csv("airlines.csv")
```

```
Rows: 14 Columns: 2
```

```
-- Column specification -----
```

```
Delimiter: ","
```

```
chr (2): IATA_CODE, AIRLINE
```

i Use ``spec()`` to retrieve the full column specification for this data.

i Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

```
airports <- read_csv(  
  "airports.csv",  
  col_select = c(  
    IATA_CODE, CITY, STATE, LATITUDE, LONGITUDE  
  )  
)
```

```
Rows: 322 Columns: 5
```

```
-- Column specification -----
```

```
Delimiter: ","
```

```
chr (3): IATA_CODE, CITY, STATE
```

```
dbl (2): LATITUDE, LONGITUDE
```

i Use ``spec()`` to retrieve the full column specification for this data.

i Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

```
estatisticas <- read_csv_chunked(  
  "flights.csv",  
  chunk_size = 1000000,  
  col_types = cols_only(  
    DESTINATION_AIRPORT = col_character(),  
    ARRIVAL_DELAY = col_double()  
  ),  
  callback = DataFrameCallback$new(function(x, pos) {  
    x |>
```

```

    filter(!grepl("^1", DESTINATION_AIRPORT)) |>
    na.omit() |>
    group_by(DESTINATION_AIRPORT) |>
    summarise(soma = sum(ARRIVAL_DELAY), n = n())
  })
)

flights <- estatisticas |>
  group_by(DESTINATION_AIRPORT) |>
  summarise(atraso_medio = sum(soma) / sum(n))

flights

```

```

# A tibble: 322 x 2
  DESTINATION_AIRPORT atraso_medio
  <chr>                <dbl>
1 ABE                    5.80
2 ABI                    4.08
3 ABQ                    5.61
4 ABR                   -3.39
5 ABY                    8.68
6 ACK                    4.91
7 ACT                   10.6
8 ACV                    9.68
9 ACY                   11.8
10 ADK                   -3.19
# i 312 more rows

```

2. Selecione a operação apropriada join para incluir, na tabela flights, as colunas CITY e STATE do objeto airports. Para executar esta tarefa:
 - a. Identifique a coluna que é a chave na tabela flights;
 - b. Identifique a coluna que é a chave na tabela airports;
 - c. Quais são os aeroportos que estão listados em flights, mas estão ausentes em airports?
 - d. Apresente o comando que combine ambas as tabelas, indicando explicitamente as chaves;
 - e. Armazene a tabela resultante no objeto flights.

```
ausentes <- flights |>
  anti_join(airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE")) |>
  select(DESTINATION_AIRPORT)

ausentes
```

```
# A tibble: 0 x 1
# i 1 variable: DESTINATION_AIRPORT <chr>
```

Nenhum aeroporto ausente.

```
flights <- flights |>
  left_join(
    airports |>
      select(IATA_CODE, CITY, STATE), by = c("DESTINATION_AIRPORT" = "IATA_CODE")
  )

flights
```

```
# A tibble: 322 x 4
  DESTINATION_AIRPORT atraso_medio CITY      STATE
  <chr>                <dbl> <chr>    <chr>
1 ABE                  5.80 Allentown PA
2 ABI                  4.08 Abilene   TX
3 ABQ                  5.61 Albuquerque NM
4 ABR                 -3.39 Aberdeen SD
5 ABY                  8.68 Albany   GA
6 ACK                  4.91 Nantucket MA
7 ACT                 10.6 Waco      TX
8 ACV                  9.68 Arcata/Eureka CA
9 ACY                 11.8 Atlantic City NJ
10 ADK                 -3.19 Adak    AK
# i 312 more rows
```

3. Quantos aeroportos cada estado possui? Apresente uma tabela ordenada de forma decrescente (no número de aeroportos).

```
aerportos_estado <- airports |>
  filter(!is.na(STATE)) |>
  group_by(STATE) |>
  summarise(TOTAL_AEROPORTOS = n()) |>
```

```

arrange(desc(TOTAL_AEROPORTOS))

print(aeropostos_estado, n = Inf)

```

```

# A tibble: 54 x 2
  STATE TOTAL_AEROPORTOS
  <chr>         <int>
1 TX             24
2 CA             22
3 AK             19
4 FL             17
5 MI             15
6 NY             14
7 CO             10
8 MN              8
9 MT              8
10 NC              8
11 ND              8
12 PA              8
13 WI              8
14 GA              7
15 IL              7
16 LA              7
17 VA              7
18 ID              6
19 WY              6
20 AL              5
21 HI              5
22 IA              5
23 MA              5
24 MO              5
25 MS              5
26 OH              5
27 OR              5
28 TN              5
29 UT              5
30 AR              4
31 AZ              4
32 IN              4
33 KS              4
34 KY              4
35 NM              4

```

36	SC	4
37	WA	4
38	NE	3
39	NJ	3
40	NV	3
41	OK	3
42	PR	3
43	SD	3
44	ME	2
45	VI	2
46	AS	1
47	CT	1
48	DE	1
49	GU	1
50	MD	1
51	NH	1
52	RI	1
53	VT	1
54	WV	1

4. Apresente um mapa representando todos os atrasos observados por aeroporto.
 - a. Carregue o pacote leaflet;
 - b. Combine os comandos leaflet, addTiles e addMarkers para a criação de um mapa básico;
 - c. Armazene o grafico em b) numa variável chamada this;
 - d. Adicione as duas linhas abaixo após a criação da variável this (apenas se você estiver usando Jupyter):

```
library(leaflet)

#| label: fig-mapa
#| fig-cap: "Mapa atraso médio por aeroporto"
#| fig-format: png

dados_mapa <- flights |>
  left_join(airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE")) |>
  filter(!is.na(LATITUDE) & !is.na(LONGITUDE))

this <- leaflet(dados_mapa) |>
  addTiles() |>
  addMarkers(
```

```
  lng = ~LONGITUDE,  
  lat = ~LATITUDE  
)
```

```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-08-31 21:32:42 -03"
```